

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Zboruri**

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Introducere și Contextul Sistemului	2
1.1	Harta de Context (Context Map)	2
2	Arhitectura Internă a Microserviciului Zboruri	3
2.1	Diagrama de Pachete (Arhitectura Hexagonală)	3
2.2	Diagrama de Clase Detaliată	4
3	Design-ul Bazei de Date (Persistența)	5
4	Modelarea Domeniului (Domain-Driven Design)	5
4.1	Aggregate Root — Clasa Zbor	5
4.2	Entity și Value Objects	5
4.3	Prezentarea instrumentului de testare utilizat	6
4.4	Descrierea testării tuturor metodelor microserviciului	6
5	Șabloane de Proiectare (Design Patterns) Utilizate	11
5.1	Șablonul Adaptor (Adapter / Wrapper)	11
5.2	Șablonul Singleton (Gestionat prin IoC)	11
6	Concluzii	11

1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale, fiecare acționând ca un microserviciu independent: *Zboruri*, *Bilete*, *Utilizatori*, *Notificări*, *Statistici* și *Export*.

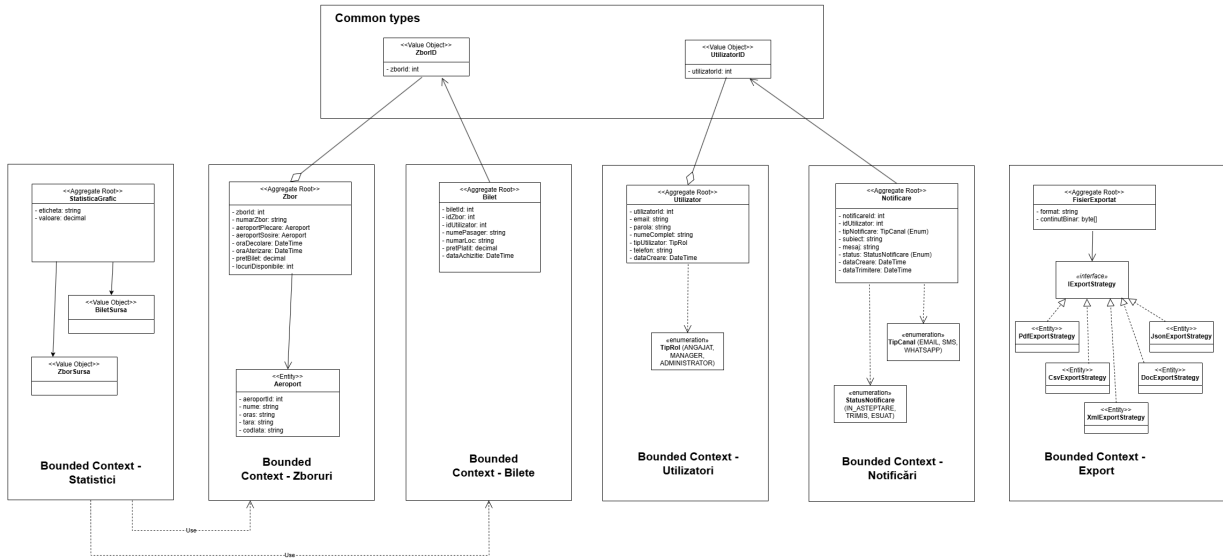


Figura 1: Harta de context a sistemului global de gestiune

Justificarea deciziei: Această decizie arhitecturală previne crearea unui monolit rigid. De exemplu, contextul **Zboruri** (vizat exclusiv în acest document) nu are cunoștințe despre pasageri sau plăți. El expune doar concepte de infrastructură aeriană (**Zbor**, **Aeroport**). Tipurile comune (precum **ZborID** și **UtilizatorID**) sunt extrase într-un pachet comun de *Value Objects* pentru a permite comunicarea între microservicii prin intermediul unor identificatori unici, fără a partaja direct tabele de baze de date. De asemenea, adăugarea ulterioară a microserviciilor **Statistici** (care acționează ca un consumator de date agregate) și **Export** (complet *stateless*) demonstrează scalabilitatea sistemului: noi funcționalități pot fi introduse fără a altera nucleul de business existent.

2 Arhitectura Internă a Microserviciului Zboruri

2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Zboruri** respectă cu strictețe principiul Inversiunii Dependențelor (*Dependency Inversion Principle*), specific Arhitecturii Hexagonale (sau Ports and Adapters).

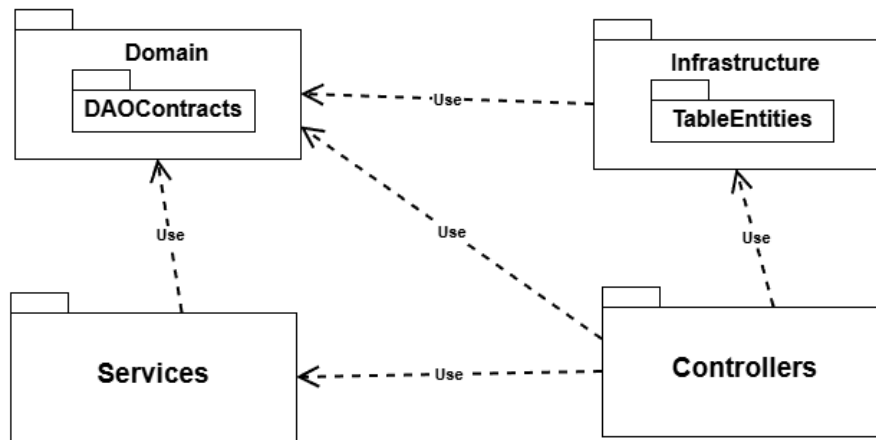


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

Justificarea deciziei: Săgețile din figura 2 indică direcția dependențelor (-Use-). Se observă clar că inima aplicației este pachetul **Domain**, care nu depinde de nimic. Pachetul **Infrastructure** (care conține tehnologii volatile precum bazele de date și entitățile ORM) depinde de Domain pentru a-i implementa contractele (DAOContracts). Pachetul **Controllers** (expunerea REST) apelează pachetul **Services**, care la rândul său orchestrează modelele din Domain. Această structură izolează logica de business de orice framework tehnic extern.

2.2 Diagrama de Clase Detaliată

Pentru a expune în mod granular structura internă, tipurile de date, vizibilitatea atributelor și comportamentele metodelor din fiecare strat, a fost mapată structura de clase a întregului modul.

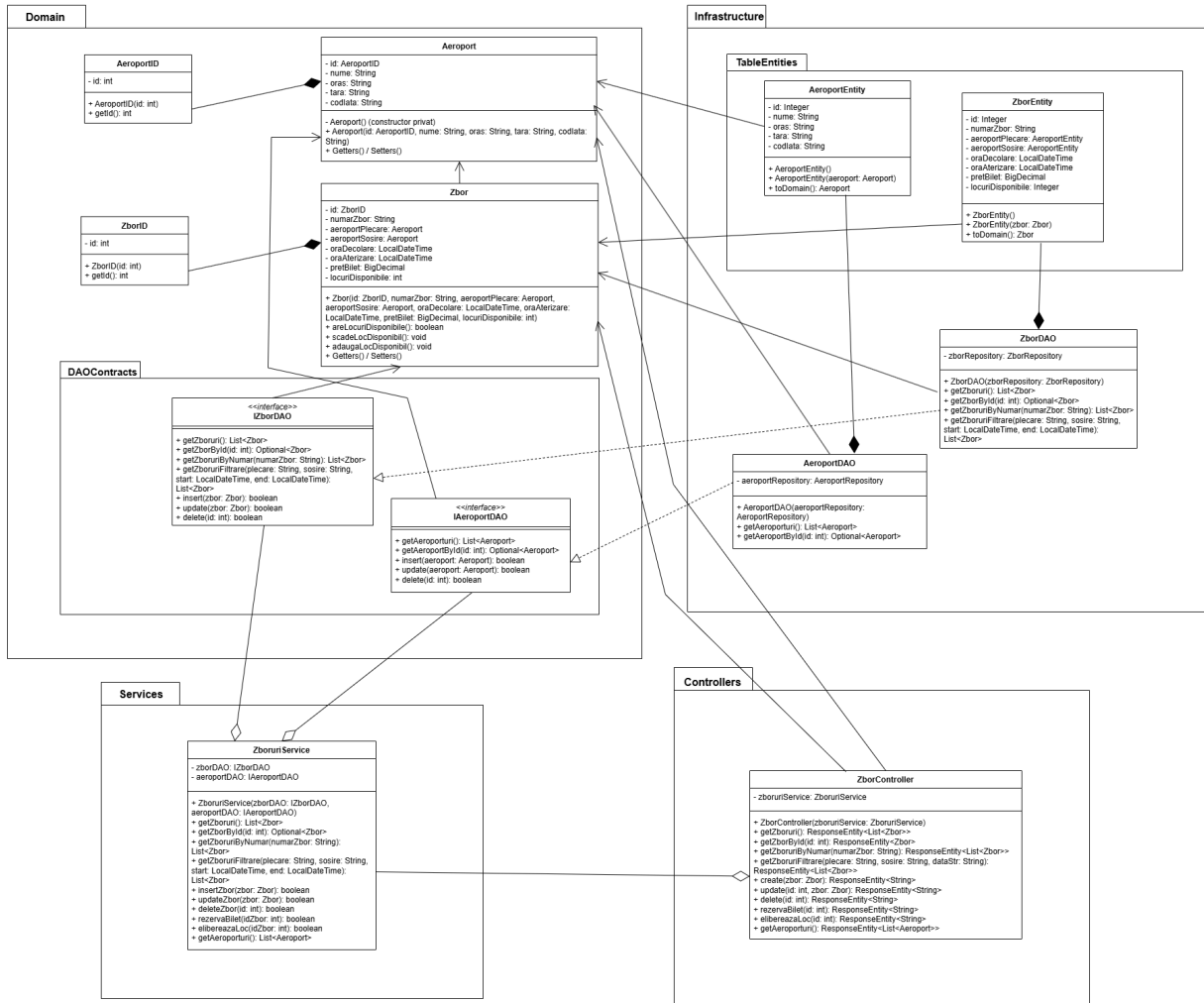


Figura 3: Diagrama de clase detaliată a microserviciului Zboruri

Justificarea deciziei: Figura 3 oferă o vedere structurală completă a microserviciului. Ea evidențiază clar asocierile direcționate generate de mecanismul de injectare a dependențelor din Spring (de la Controller la Service, și de la Service la DAO-uri), relațiile de compoziție dintre entitățile de domeniu și identificatorii lor imutabili de tip Value Object, precum și realizarea/implementarea contractelor de date prin straturile izolate din infrastructură.

3 Design-ul Bazei de Date (Persistența)

Pentru salvarea datelor s-a optat pentru un model relațional (SQL), transpus în baza de date prin intermediul ORM-ului Hibernate.

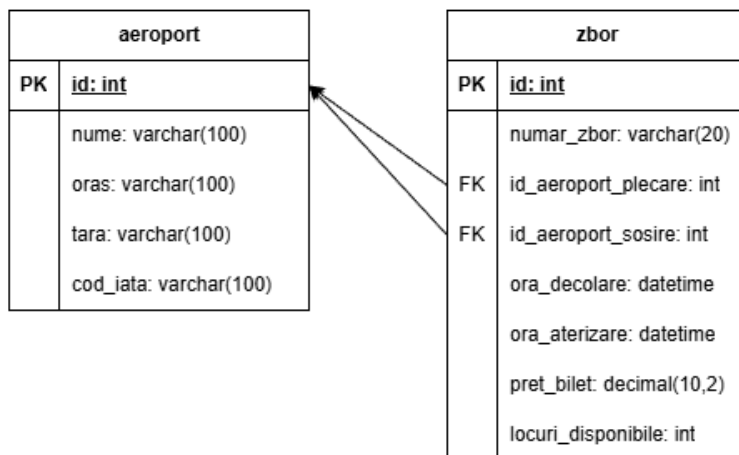


Figura 4: Diagrama Entitate-Relație (ERD) pentru modulul Zboruri

Justificarea deciziei: Figura 4 ilustrează relația de tip "Mulți-la-Unu" (Many-to-One) între un zbor și aeroporturile sale. Deoarece un zbor necesită o rută clară, tabela **zbor** deține două chei străine (Foreign Keys): **id_aeroport_plecare** și **id_aeroport_sosire**, ambele referențiind cheia primară din tabela **aeroport**. Această normalizare asigură consistența datelor: detaliile unui aeroport (ex: schimbarea numelui) se actualizează într-un singur loc, reflectându-se instantaneu în toate zborurile asociate.

4 Modelarea Domeniului (Domain-Driven Design)

În interiorul nucleului **Domain**, am transpus logica agenției aeriene folosind tactici DDD.

4.1 Aggregate Root — Clasa Zbor

Clasa **Zbor** funcționează ca un *Aggregate Root*. Ea garantează că nicio modificare nu încalcă regulile interne de business:

- **Validarea rutei:** Un avion nu poate decola și ateriza în același loc ($id_{plecare} \neq id_{sosire}$).
- **Validarea capacității:** Numărul maxim de locuri este plafonat strict la 90, iar valoarea nu poate deveni negativă.
- **Validarea cronologică:** Ora de decolare trebuie să preceadă strict ora de aterizare.

4.2 Entity și Value Objects

Aeroport reprezintă o **Entitate** (Entity) subordonată agregatului **Zbor**, identificată univoc indiferent de atributele ei. ID-urile claselor (**ZborID**, **AeroportID**) au fost modelate ca **Value Objects** pentru a asigura *Strong Typing* (Tipizare Puternică), prevenind astfel erorile de inversare a parametrilor numerici la apelul funcțiilor.

4.3 Prezentarea instrumentului de testare utilizat

Pentru documentarea interactivă și testarea endpoint-urilor expuse de microservicii, s-a utilizat **Swagger UI** (prin specificația OpenAPI).

Integrarea Swagger UI a reprezentat un pas esențial în faza de dezvoltare și testare. Deoarece modulele aplicației (Zbor, Bilete, Utilizatori, Notificări) rulează ca entități independente, Swagger a oferit un contract clar al API-urilor RESTful dezvoltate. Această tehnologie a permis vizualizarea și interacțiunea directă cu microserviciul dintr-o interfață web, facilitând execuția cererilor HTTP fără a fi necesară finalizarea în prealabil a aplicației client din JavaFX.

4.4 Descrierea testării tuturor metodelor microserviciului

Testarea metodelor expuse de **ZborController** a vizat validarea întregului flux de date, de la request-ul HTTP, trecând prin logica de business din Service, până la modificarea bazei de date. Conform contractului API vizualizat în Swagger UI, au fost testate individual toate cele 8 endpoint-uri implementate:

- **GET /api/zbor:** S-au efectuat apeluri pentru a verifica returnarea listei complete de zboruri cu un status 200 OK. S-a verificat corectitudinea serializării JSON pentru obiectele agregate (ex. **Aeroport**) și datele calendaristice.

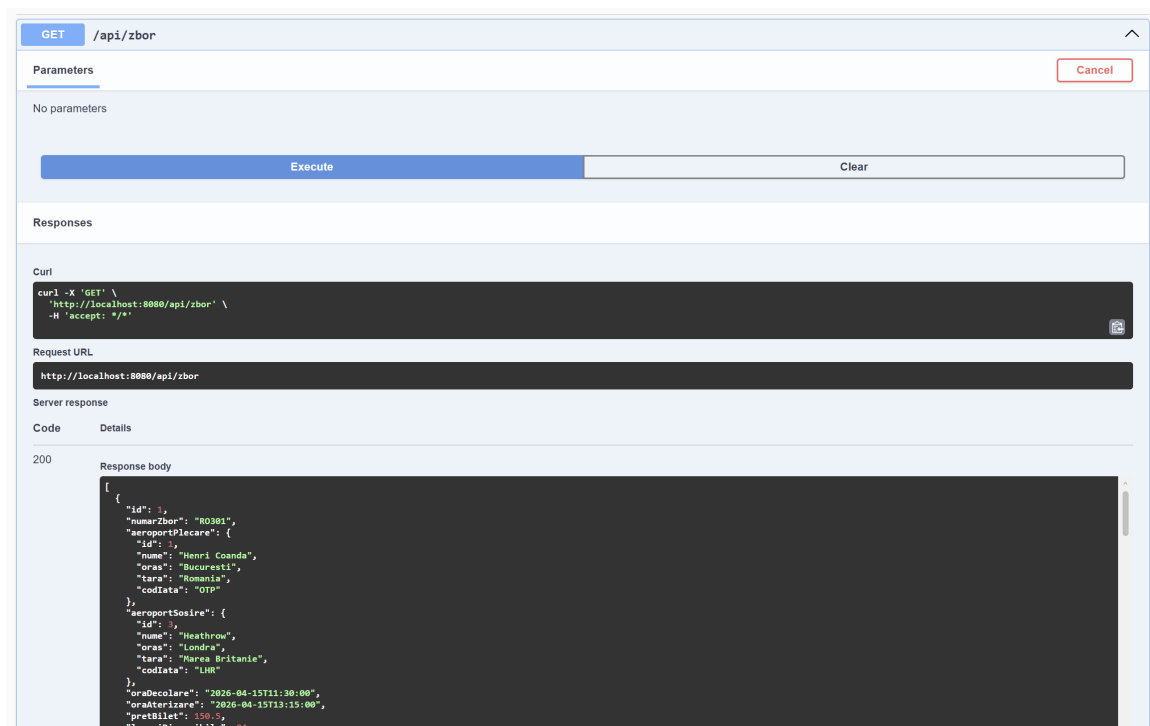


Figura 5: Testare GET /api/zbor - preluarea listei complete de zboruri

- **PUT /api/zbor:** S-a testat capacitatea de a actualiza detaliile unui zbor existent, transmițând un obiect JSON modificat și verificând salvarea corectă (**update**) în baza de date, alături de interceptarea erorilor de business pe datele noi.



Figura 6: Testare PUT /api/zbor - parametrii trimiși și confirmarea actualizării

- **POST /api/zbor:** S-a transmis un payload JSON valid pentru a verifica inserarea unui zbor nou (201 Created). Tot aici s-au testat intensiv validările de business, forțând erori (ex: ID aeroport plecare identic cu cel de sosire sau o capacitate de peste 90 de locuri) pentru a confirma primirea răspunsului 400 Bad Request.

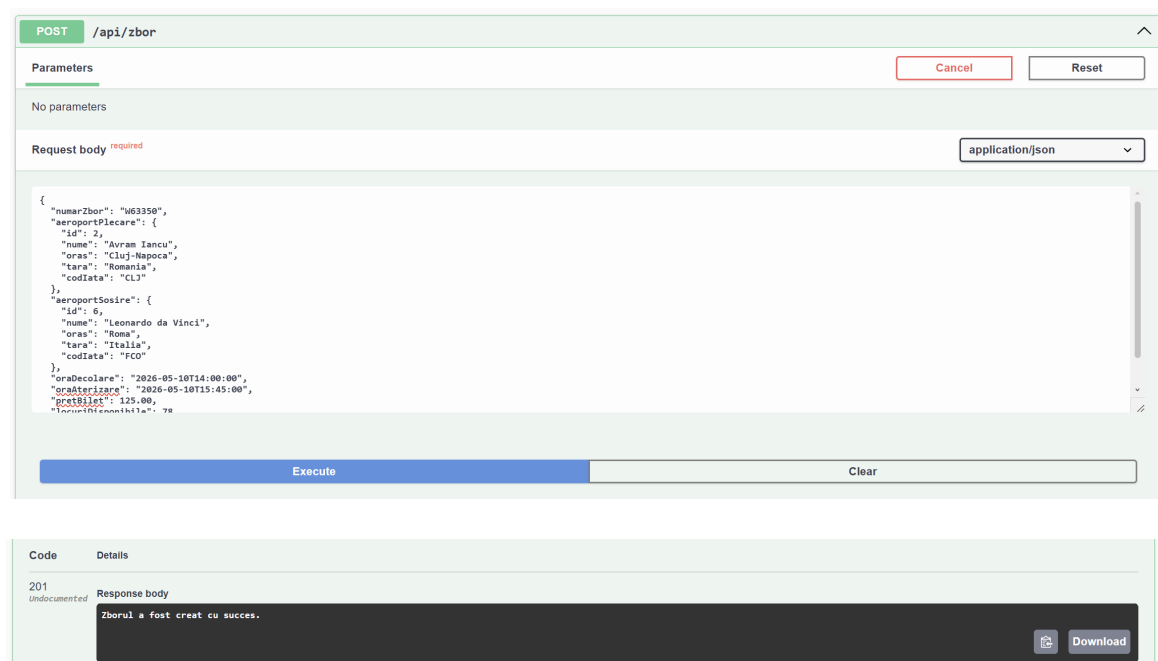
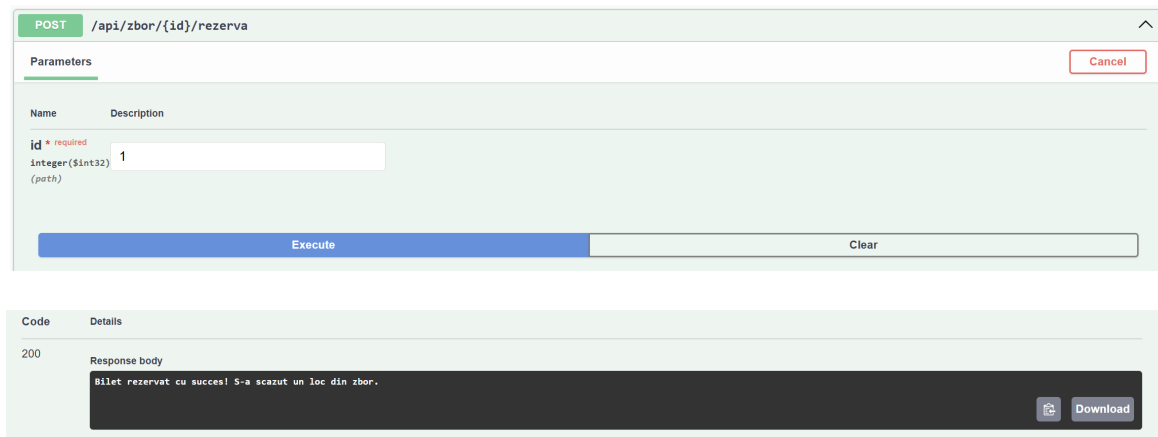


Figura 7: Testare POST /api/zbor - crearea unui zbor nou

- **POST /api/zbor/{id}/rezerva:** S-a testat direct logica de business pentru vânzarea biletelor. Apelarea acestui endpoint a confirmat descreșterea locurilor disponibile cu o unitate în baza de date. S-a forțat scenariul unui zbor cu 0 locuri, validând că sistemul aruncă excepția „SOLD OUT”.



POST /api/zbor/{id}/rezerva

Parameters

Name	Description
id * required integer(\$int32) (path)	1

Execute Clear

Code Details

200

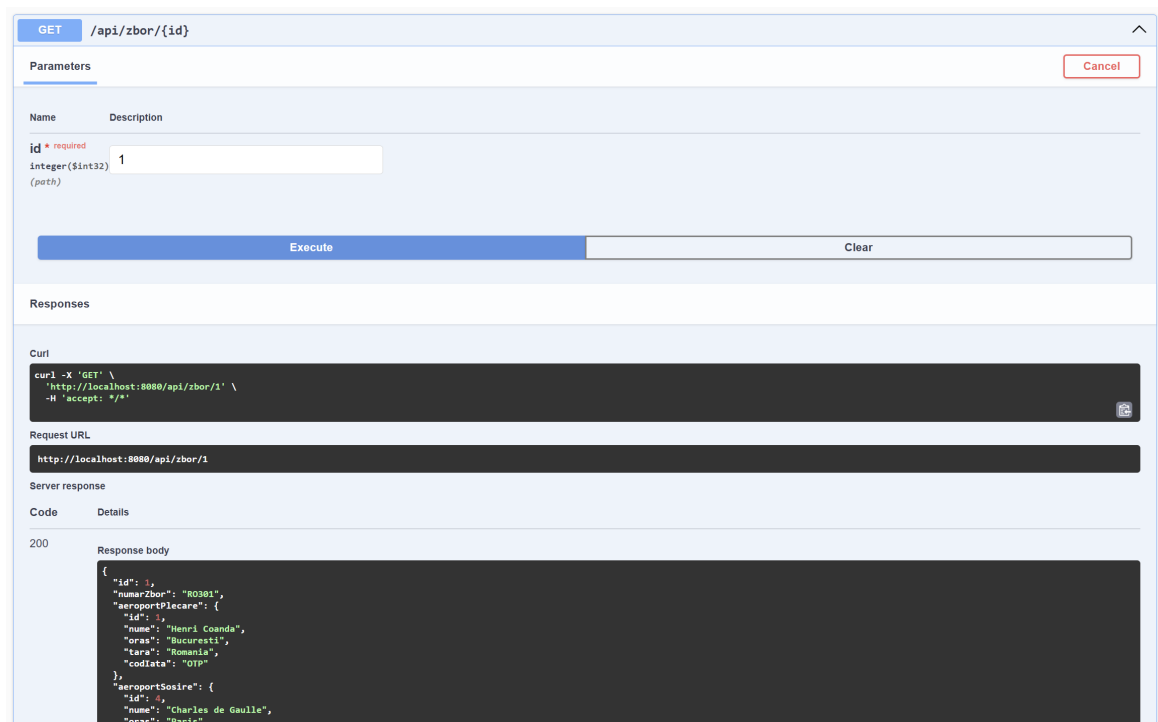
Response body

```
Bilet rezervat cu succes! S-a sczut un loc din zbor.
```

Download

Figura 8: Testare POST /api/zbor/{id}/rezerva - rezervarea unui bilet

- **GET /api/zbor/{id}:** S-a verificat preluarea detaliilor unui zbor specific folosind identificatorul său unic (PathVariable), validând întoarcerea obiectului corect sau a unui status de tip Not Found dacă ID-ul nu există.



GET /api/zbor/{id}

Parameters

Name	Description
id * required integer(\$int32) (path)	1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/zbor/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/zbor/1
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "numarZbor": "R0301",
  "aeroportPlecare": {
    "id": 1,
    "nume": "Henri Coanda",
    "oras": "Bucuresti",
    "tara": "Romania",
    "codIata": "OTP"
  },
  "aeroportSosire": {
    "id": 4,
    "nume": "Charles de Gaulle",
    "oras": "Paris"
  }
}
```

Figura 9: Testare GET /api/zbor/{id} - preluarea detaliilor unui singur zbor

- **DELETE /api/zbor/{id}**: S-a testat eliminarea logică sau fizică a unui zbor din sistem pe baza ID-ului, verificând ștergerea efectivă din tabelul bazei de date.

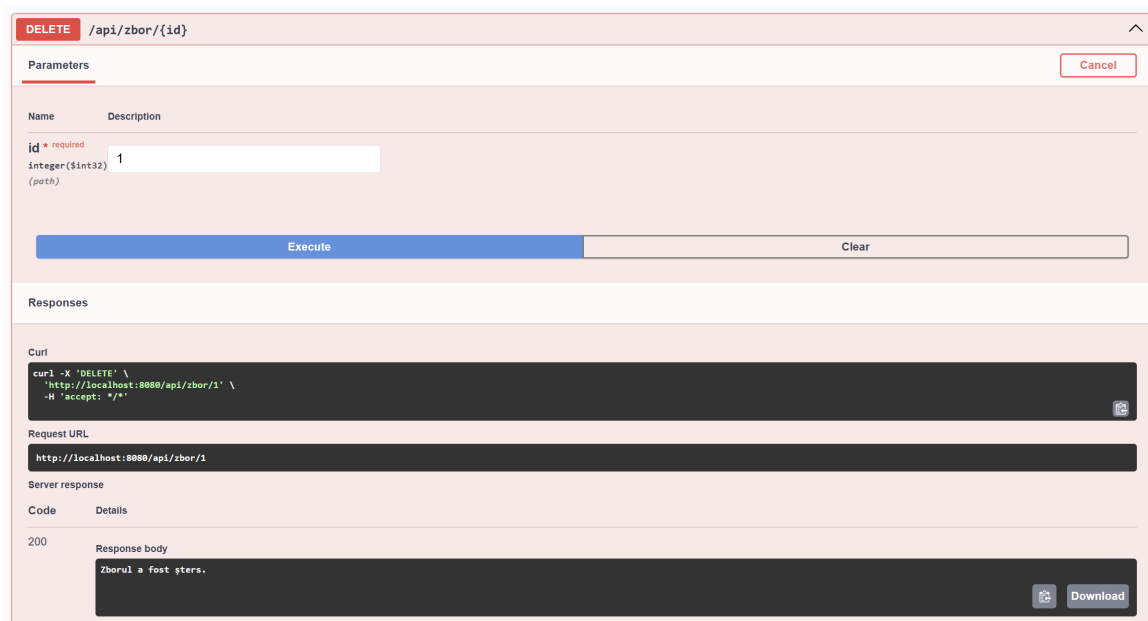


Figura 10: Testare DELETE /api/zbor/{id} - ștergerea unui zbor existent

- **GET /api/zbor/numar/{numarZbor}**: S-a validat interogarea personalizată din repository pentru căutarea zborurilor după numărul alocat, funcționalitate esențială pentru cazurile de utilizare ale managerului.

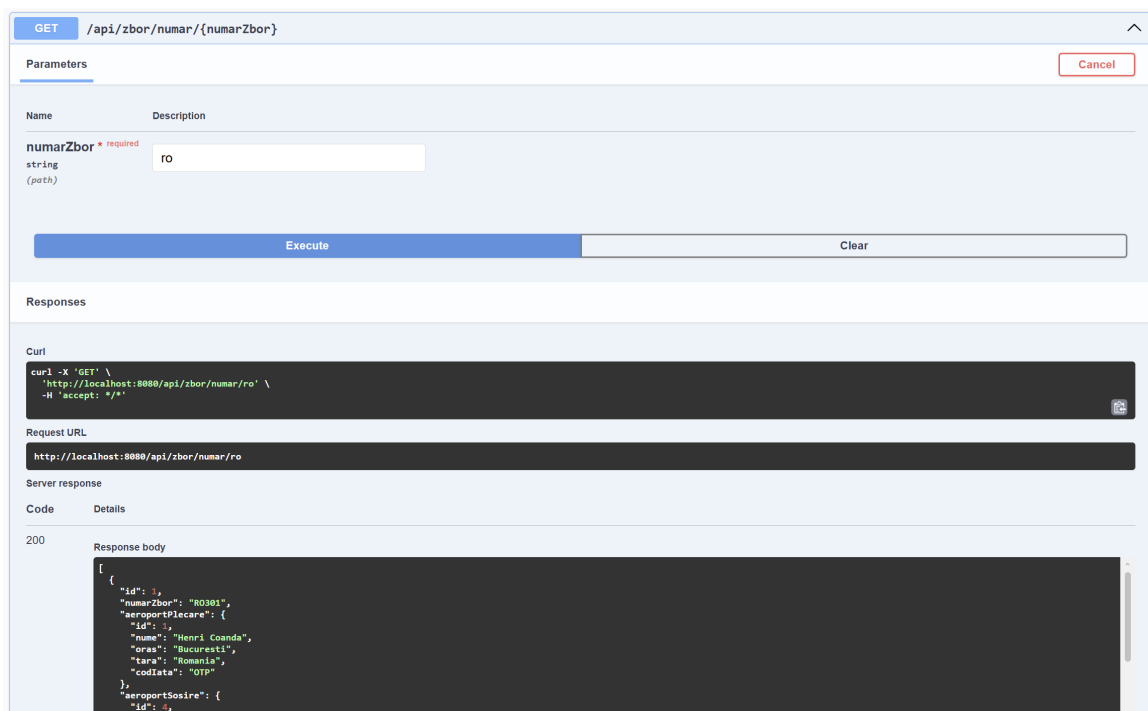


Figura 11: Testare GET /api/zbor/numar/{numarZbor} - căutarea după numărul zborului

- GET /api/zbor/filtrare: S-au trimis parametrii de query necesari (idAeroportPlecare, idAeroportSosire, data). S-a confirmat că backend-ul calculează corect începutul și sfârșitul zilei selectate (startOfDay, endOfDay) și filtrează exact înregistrările corespunzătoare.

The screenshot shows a REST client interface for the endpoint `GET /api/zbor/filtrare`. The parameters section contains three required fields: `idAeroportPlecare` (integer, value 1), `idAeroportSosire` (integer, value 3), and `data` (string, value 2026-04-15T08:30:00). Below the parameters are 'Execute' and 'Clear' buttons. The responses section shows the generated curl command, the request URL, and the server response. The server response is a 200 status code with a JSON body: `{ "id": 0, "numarZbor": "R0305" }`.

Name	Description
<code>idAeroportPlecare</code> * required integer(\$int32) (query)	1
<code>idAeroportSosire</code> * required integer(\$int32) (query)	3
<code>data</code> * required string(\$date-time) (query)	2026-04-15T08:30:00

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  http://localhost:8080/api/zbor/filtrare?idAeroportPlecare=1&idAeroportSosire=3&data=2026-04-15T08%3A30%3A00' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/zbor/filtrare?idAeroportPlecare=1&idAeroportSosire=3&data=2026-04-15T08%3A30%3A00
```

Server response

Code	Details
200	Response body <pre>{ "id": 0, "numarZbor": "R0305", }</pre>

Figura 12: Testare GET /api/zbor/filtrare - filtrarea avansată a zborurilor

5 Șabloane de Proiectare (Design Patterns) Utilizate

5.1 Șablonul Adaptor (Adapter / Wrapper)

Pentru a realiza comunicarea între stratul de **Domain** și cel de **Infrastructure** (vezi Figura 2), fără a contamina nucleul de business cu detalii tehnice legate de baza de date relațională, am integrat pattern-ul Adaptor.

Explicație și Justificare: În cadrul arhitecturii noastre detaliate în Figura 3, interfețele **IZborDAO** și **IAeroportDAO** reprezintă **Porturile** expuse în interiorul domeniului pur. Clasele concrete **ZborDAO** și **AeroportDAO** din infrastructură implementează aceste porturi, acționând ca **Adaptoare**. Rolul lor critic este de a prelua modelele de domeniu și de a le traduce/mapa în entități compatibile cu baza de date (ex: **ZborEntity**) pentru a putea fi persistate prin intermediul Spring Data JPA. Acest mecanism asigură că nicio modificare tehnică adusă tabelelor SQL sau structurii JPA nu va afecta regulile centrale de business.

5.2 Șablonul Singleton (Gestionat prin IoC)

Singleton este un design pattern creational care asigură faptul că o clasă are o singură instanță globală pe parcursul rulării aplicației. În acest proiect, implementarea șablonului este delegată complet containerului *Inversion of Control (IoC)* din framework-ul Spring.

Explicație și Justificare: Clasele marcate cu stereotipuri Spring, precum **@RestController** (**ZborController**), **@Service** (**ZboruriService**), **@Component** (**ZborDAO**) sau **@Repository**, sunt înregistrate automat ca *Singleton Beans*. La pornirea aplicației, Spring creează o singură instanță în memorie din **ZboruriService**. Atunci când mii de utilizatori accesează endpoint-urile pentru a rezerva bilete, toți vor folosi aceeași instanță a serviciului, făcând aplicația extrem de eficientă din punct de vedere al resurselor (memorie și procesor).

Serviciile și Adaptoarele noastre sunt proiectate să fie *stateless* (nu stochează date specifice unui client în atributele lor), ceea ce le face perfect sigure pentru utilizarea partajată și concurentă (thread-safe) specifică pattern-ului Singleton.

6 Concluzii

Arhitectura microserviciului **Zboruri** demonstrează o separare matură a preocupărilor (Separation of Concerns). Prin combinarea Hărților de Context pentru delimitarea funcționalităților, a pattern-ului Adaptor pentru izolarea bazei de date și a puterii containerului Spring (Singleton/DI), rezultă un modul software robust, extensibil și extrem de ușor de testat în mod izolat.